

Project Proposal & Development Plan

Real-Time Cyber Anomaly Detection System

Version: 2.0 **Date:** 2025-12-12 **Team:** Francesca Craievich (PM), Lucas Jakin (ML Engineer), Francesco Rumiz (Data Scientist)

1. Executive Summary

1.1 Project Overview

The Real-Time Cyber Anomaly Detection System is a machine learning-based platform designed to identify malicious network traffic patterns and security threats in real-time. By analyzing Suricata IDS logs (selected for complete feature set) and normal network traffic, the system employs One-Class SVM to distinguish between benign and anomalous network behavior.

1.2 Objectives

- Develop an ML-based anomaly detection system for network security
- Achieve high detection accuracy with minimal false positives
- Create a modular, maintainable codebase following MLOps best practices
- Deploy real-time dashboards for security analysts and ML engineers

1.3 Business Value

- **Proactive Threat Detection:** Identify security threats before they cause damage
- **Reduced Response Time:** Real-time anomaly detection enables immediate action
- **Cost Efficiency:** Automated detection reduces manual security monitoring effort
- **Research Contribution:** Advances understanding of cyber attack patterns

1.4 Success Metrics

Metric	Target	Achieved
Detection Rate (TPR)	≥ 85%	90.7% ✓
False Positive Rate (FPR)	≤ 15%	~10% ✓
Processing Latency	≤ 100ms per event	✓
System Uptime	≥ 99%	✓
Throughput	≥ 1,000 events/second	✓

2. Project Scope

2.1 In Scope

- Data collection from TPOT honeypot platform
- Feature engineering pipeline for network traffic logs
- One-Class SVM model training and deployment
- Anomaly detection inference engine
- Real-time dashboards (anomaly alerts + ML monitoring)
- REST API for predictions and statistics
- Documentation and testing infrastructure
- CI/CD pipeline for automated deployment
- Drift detection and model monitoring

2.2 Out of Scope (Future Phases)

- Real-time stream processing (Kafka/MQTT integration)
- Multi-model ensemble approaches
- Cloud deployment infrastructure (AWS/Azure/GCP)
- Advanced alert notification systems (email, Slack)
- User authentication and access control

2.3 Deliverables

ID	Deliverable	Description	Owner	Status
D1	Data Collection Pipeline	Scripts to extract and preprocess honeypot logs	Data Scientist	✓ Completed
D2	Feature Engineering Module	Statistical feature extraction from network traffic	Data Scientist	✓ Completed
D3	ML Model	Trained One-Class SVM anomaly detection model	ML Engineer	✓ Completed
D4	Inference Engine	Real-time classification service with Flask API	All Team	✓ Completed
D5	Testing Suite	Unit and integration tests (pytest)	All Team	✓ Completed
D6	Documentation	Technical documentation (MkDocs)	Project Manager	✓ Completed
D7	CI/CD Pipeline	GitHub Actions workflows	Project Manager	✓ Completed
D8	System Specification	SSD document	All Team	✓ Completed
D9	Dashboards	Streamlit dashboards (alerts + monitoring)	Software Developer	✓ Completed
D10	Monitoring Stack	Prometheus + Grafana setup	Software Developer	✓ Completed

3. Project Timeline & Milestones

3.1 Project Evolution Timeline

The project evolved through several distinct phases, each marked by key technical decisions and milestones:

Period	Milestone	Key Deliverable	Commit Reference
Oct 2024	Project Initialization	Repository setup, documentation structure	e951421 Initial commit
Oct 2024	Data Collection	T-Pot honeypot deployment on GCP, 148K events collected	Infrastructure setup
Oct 2024	Data Engineering Foundation	DataFrame initializer classes	5954d85 DfInitializer class
Nov 2024	Feature Analysis	Column mapping, honeypot evaluation	9f2f925 log_formatting
Nov 2024	Data Source Decision	Selection of Suricata as primary source	Feature compatibility analysis
Nov 2024	Initial Model	One-Class SVM v2 with contamination parameter	808c4ff One Class SVM v2
Nov 2024	Model Optimization	Grid Search hyperparameter tuning	380b450 GridSearch
Nov 2024	Model Refinement	Hyperparameter tuning, multiclass distinction	7ceeac3 model v4
Dec 2024	Drift Detection v1	Initial ADWIN implementation	feff13a Drift Detection first version
Dec 2024	Drift Detection v2	Anomaly rate-based drift detection	ccb1328 Drift implementation v2
Dec 2024	Dashboard & API	Flask REST API + Streamlit dashboard	1c88ac8 Streamlit dashboard with Flask API
Dec 2024	Monitoring Stack	Prometheus + Grafana integration	2e85927 Add Grafana and Prometheus monitoring
Dec 2024	Cloud Deployment	Streamlit Cloud deployment	01c6e80 , 8934f37 Streamlit Cloud links
Dec 2024	Code Quality	Black, isort formatting, linting fixes	5878ba7 , a4d9fc8 Format and lint
Dec 2024	Security Hardening	SECURITY.md, CodeQL integration	378b5f5 Security improvements

3.2 Development Phases

Phase 1: Planning & Requirements

- Requirements gathering and analysis
- System architecture design
- Technology stack selection
- Documentation setup

Phase 2: Data Pipeline Development

- Honeypot log collection from TPOT
- Data preprocessing and cleaning
- Feature engineering implementation
- DataFrame initialization modules

Phase 3: Model Development

- One-Class SVM implementation
- Model training and hyperparameter tuning
- Performance evaluation and validation
- Model persistence mechanism

Phase 4: Integration & Testing

- Component integration
- Unit and integration testing
- Performance benchmarking
- Bug fixing and optimization

Phase 5: Documentation & Deployment

- Technical documentation completion
- CI/CD pipeline setup
- Dashboard deployment to Streamlit Cloud
- Final testing and validation

3.3 Key Milestones

Milestone	Status	Deliverables
M1: Project Kickoff	✓ Completed	Requirements doc, team roles
M2: Data Collection Complete	✓ Completed	Honeypot logs, normal traffic dataset
M3: Feature Engineering Ready	✓ Completed	Feature extraction pipeline
M4: Model Training Complete	✓ Completed	Trained OCSVM model (90.7% TPR)
M5: Testing Infrastructure	✓ Completed	Pytest suite (116 tests), CI/CD pipeline
M6: Documentation Published	✓ Completed	MkDocs site, SSD, proposal docs
M7: Dashboards Deployed	✓ Completed	Streamlit Cloud apps, monitoring stack

4. Work Breakdown Structure (WBS)

4.1 Level 1: Project Phases

```
1.0 Real-Time Cyber Anomaly Detection System
├── 1.1 Project Management
├── 1.2 Data Engineering
├── 1.3 Machine Learning
├── 1.4 Testing & Quality Assurance
├── 1.5 Documentation
└── 1.6 Deployment & Operations
```

4.2 Level 2: Detailed Tasks

1.1 Project Management

- 1.1.1 Requirements gathering and analysis
- 1.1.2 Team coordination and communication
- 1.1.3 Milestone tracking and reporting
- 1.1.4 Risk management
- 1.1.5 Documentation coordination

1.2 Data Engineering

- 1.2.1 Suricata IDS log collection from TPOT
- 1.2.2 Normal traffic dataset acquisition (CICIDS/ISCX benign traffic)
- 1.2.3 JSON parsing and preprocessing
- 1.2.4 Data validation and quality checks
- 1.2.5 Feature extraction pipeline
 - 1.2.5.1 Suricata feature extraction
 - 1.2.5.2 Normal traffic feature extraction
 - 1.2.5.3 Aggregation functions
 - 1.2.5.4 Precalculation utilities
- 1.2.6 DataFrame initialization modules

1.3 Machine Learning

- 1.3.1 Algorithm selection and justification
- 1.3.2 One-Class SVM implementation
- 1.3.3 Hyperparameter tuning (Grid Search)
- 1.3.4 Model training pipeline
- 1.3.5 Model evaluation and validation
- 1.3.6 Model persistence (pickle/joblib)
- 1.3.7 Inference engine development
- 1.3.8 Drift detection implementation

1.4 Testing & Quality Assurance

- 1.4.1 Unit test development (pytest)

- 1.4.2 Integration testing
- 1.4.3 Performance testing
- 1.4.4 Code quality checks (Black, isort, flake8)

1.5 Documentation

- 1.5.1 MkDocs setup and configuration
- 1.5.2 System Specification Document (SSD)
- 1.5.3 Project Proposal & Development Plan
- 1.5.4 Operational Governance Document
- 1.5.5 README and quick start guide

1.6 Deployment & Operations

- 1.6.1 Environment setup (requirements.txt)
- 1.6.2 Dashboard development (Streamlit)
- 1.6.3 API development (Flask)
- 1.6.4 Streamlit Cloud deployment
- 1.6.5 Monitoring stack setup (Prometheus/Grafana)

5. Sprint Planning & Agile Methodology

5.1 Methodology Framework

The project adopted **CRISP-DM** (Cross-Industry Standard Process for Data Mining) as the guiding methodology framework, integrated with Agile/Scrum practices:

CRISP-DM Phase	Sprint Focus	Key Deliverables
Business Understanding	Sprint 1	Requirements, success metrics, architecture design
Data Understanding	Sprint 1-2	EDA, data quality assessment, honeypot analysis
Data Preparation	Sprint 2-3	Feature engineering, data pipeline, dataset creation
Modeling	Sprint 3-4	OCSVM implementation, hyperparameter tuning
Evaluation	Sprint 4-5	Performance validation, drift detection
Deployment	Sprint 5-6	Flask API, Streamlit dashboards, Streamlit Cloud
Monitoring	Ongoing	Prometheus/Grafana, ADWIN drift detection

CRISP-DM was preferred over KDD (Knowledge Discovery in Databases) due to its iterative nature and explicit focus on deployment and monitoring phases, which align with the project's MLOps objectives.

5.2 Agile Implementation

The team adopted Agile methodology with Scrum elements adapted for a three-person development team.

Sprint Duration: 1-2 weeks, adjusted based on deliverable complexity

Sprint Ceremonies:

Ceremony	Frequency	Duration	Purpose
Sprint Planning	Start of sprint	1-2 hours	Define scope, assign tasks, estimate effort
Daily Standup	As needed	15 minutes	Sync on progress, identify blockers (via Teams)
Sprint Review	End of sprint	1 hour	Demo completed work, gather feedback
Sprint Retrospective	End of sprint	30 minutes	Reflect on process, identify improvements

Communication Channels:

Type	Tool	Purpose
Synchronous	Microsoft Teams	Video calls, sprint ceremonies, pair programming
Asynchronous	GitHub Issues	Task tracking with acceptance criteria
Asynchronous	GitHub PRs	Code review comments and discussions
Asynchronous	WhatsApp	Quick team coordination

Project Management Tools:

Tool	Purpose	Status
GitHub Issues	Task tracking, bug reporting, feature requests	Active
Trello	Kanban board visualization	Active
Pull Requests	Code review, CI validation gate	Active
Milestones	Sprint goal tracking	Active

5.3 Sprint Details

Sprint 1: Foundation

- Goal: Establish project infrastructure and data collection
- User Stories:
 - As a data scientist, I need to access honeypot logs so that I can analyze attack patterns
 - As a team member, I need project documentation so that I understand system requirements
- Deliverables: Project repository, raw log data, README

Sprint 2: Data Pipeline

- Goal: Build robust data processing pipeline
- User Stories:
 - As a data scientist, I need clean datasets so that I can train accurate models
 - As a developer, I need modular code so that I can maintain the system
- Deliverables: Data ingestion modules, preprocessed datasets

Sprint 3: Feature Engineering

- Goal: Extract meaningful features from network traffic

- User Stories:
 - As an ML engineer, I need statistical features so that I can train the model
 - As a data scientist, I need feature documentation so that I understand the data
- Deliverables: Feature engineering module, feature documentation

Sprint 4: Model Development

- Goal: Train and validate One-Class SVM model
- User Stories:
 - As an ML engineer, I need a trained model so that I can detect anomalies
 - As a security analyst, I need high accuracy so that I can trust the alerts
- Deliverables: Trained model (90.7% TPR, 10% FPR, F1=0.885)

Sprint 5: Testing & CI/CD

- Goal: Ensure code quality and automated deployment
- User Stories:
 - As a developer, I need automated tests so that I can prevent regressions
 - As a team lead, I need CI/CD so that deployment is reliable
- Deliverables: Test suite, CI/CD pipeline, deployed documentation

Sprint 6: Dashboards & Deployment

- Goal: Deploy user-facing dashboards
 - User Stories:
 - As a security analyst, I need a dashboard to view alerts
 - As an ML engineer, I need to monitor model health
 - Deliverables: Streamlit dashboards, Prometheus/Grafana stack
-

6. Definition of Done (DoD)

A task is considered complete when ALL criteria are met:

6.1 Code Criteria

- ☐ Code is written and follows Python best practices (PEP 8)
- ☐ Code is formatted with Black and isort
- ☐ Code passes flake8 linting checks
- ☐ Code is reviewed by at least one team member
- ☐ No hardcoded values (use configuration)
- ☐ Error handling is implemented

6.2 Testing Criteria

- ☐ Unit tests written and passing
- ☐ Test coverage $\geq 40\%$ for new code (Codecov threshold)
- ☐ No regression in existing tests
- ☐ CI/CD pipeline passes all checks

6.3 Documentation Criteria

- ☐ Code is documented with clear comments where needed
- ☐ README updated (if applicable)
- ☐ MkDocs documentation updated (if applicable)

6.4 Quality Criteria

- ☐ No critical bugs or security issues
- ☐ Performance requirements met
- ☐ Code is merged to main branch
- ☐ Feature branch is deleted after merge

7. Definition of Ready (DoR)

A task can be started when ALL criteria are met:

7.1 Clarity Criteria

- ☐ User story or task description is clear and unambiguous
- ☐ Acceptance criteria are defined and testable
- ☐ Dependencies are identified and resolved (or explicitly noted)
- ☐ Technical approach has been discussed with the team

7.2 Resources Criteria

- ☐ Required data/resources are available
- ☐ Team member has capacity
- ☐ Necessary tools are accessible
- ☐ Blocking issues are resolved

7.3 Planning Criteria

- ☐ Estimated effort is assigned (S/M/L)
- ☐ Priority is assigned
- ☐ Owner is identified

8. Resource Allocation

8.1 Team Structure

Role	Team Member	Responsibilities
Project Manager	Francesca Craievich	Project planning, coordination, documentation, milestone tracking, stakeholder communication, infrastructure setup
ML Engineer	Lucas Jakin	Model development, drift detection, performance optimization

Role	Team Member	Responsibilities
Data Scientist	Francesco Rumiz	Data analysis, feature engineering, model validation, data quality assurance

8.2 Tools & Infrastructure

Development Tools:

- Python 3.11+
- Visual Studio Code / PyCharm
- Git version control
- GitHub for collaboration

Libraries & Frameworks:

- pandas, numpy: Data processing
- scikit-learn: Machine learning
- pytest: Testing
- mkdocs: Documentation
- ijson: Streaming JSON parsing
- flask, streamlit: Dashboard & API
- prometheus-client: Metrics

Infrastructure:

- Local development machines
- GitHub repository: <https://github.com/francescacraievich/real-time-cyber-anomaly-detection>
- GitHub Actions: CI/CD
- GitHub Pages: Documentation hosting
- Streamlit Cloud: Dashboard hosting

8.3 Budget Considerations

This is an academic/research project with zero financial budget. All tools and services used are open-source or free-tier offerings.

9. Risk Management

9.1 Risk Register

Risk ID	Description	Impact	Probability	Mitigation Strategy	Owner
R-01	Team member unavailability	High	Medium	Cross-training, documentation	PM
R-02	Data quality issues	High	Medium	Validation pipeline, quality checks	DS

Risk ID	Description	Impact	Probability	Mitigation Strategy	Owner
R-03	Model performance below target	High	Low	Hyperparameter tuning, algorithm exploration	ML
R-04	Technical debt accumulation	Medium	Medium	Code reviews, refactoring sprints	All
R-05	Scope creep	Medium	High	Strict scope management, backlog prioritization	PM
R-06	Integration challenges	Medium	Low	Early integration testing, modular design	ML
R-07	Documentation lag	Low	High	Continuous documentation, DoD enforcement	PM

9.2 Dependency Management

External Dependencies:

- TPOT honeypot platform availability
- Python package ecosystem stability
- GitHub service uptime
- Streamlit Cloud availability

Internal Dependencies:

- Data pipeline must complete before model training
- Feature engineering depends on data collection
- Testing requires completed implementations
- Dashboards depend on Flask API

10. Communication Plan

10.1 Team Meetings

Meeting	Frequency	Duration	Agenda
Daily Standup	As needed	15 min	Progress, blockers, plans
Sprint Planning	Bi-weekly	2 hours	Backlog review, task assignment
Sprint Review	Bi-weekly	1 hour	Demo, feedback
Sprint Retrospective	Bi-weekly	30 min	Process improvements

10.2 Communication Channels

Channel	Purpose	Response Time
---------	---------	---------------

Channel	Purpose	Response Time
Microsoft Teams	Meetings, urgent issues	Immediate
GitHub Issues	Task tracking	Within 24 hours
GitHub PRs	Code review	Within 48 hours
WhatsApp	Quick coordination	Within hours

10.3 Documentation Repository

All project documentation is version-controlled in the docs/ folder and published to GitHub Pages at: <https://francescakraievich.github.io/real-time-cyber-anomaly-detection/>

11. Success Criteria

The project is considered successful when:

11.1 Technical Criteria

- ☒ One-Class SVM model achieves $\geq 85\%$ TPR with $\leq 15\%$ FPR ✓ (90.7% TPR, 10% FPR)
- ☒ System processes $\geq 1,000$ events/second ✓
- ☒ Inference latency $\leq 100\text{ms}$ per event ✓
- ☒ All unit tests pass ✓ (116 tests passing)
- ☒ CI/CD pipeline is operational ✓ (GitHub Actions + Codecov)

11.2 Delivery Criteria

- ☒ All deliverables (D1-D12) are completed ✓
- ☒ Documentation is published and accessible ✓ (GitHub Pages)
- ☒ Code is merged to main branch ✓
- ☒ System is deployable from repository ✓
- ☒ Dashboards are accessible ✓ (Streamlit Cloud)

11.3 Documentation Criteria

- ☒ System Specification Document is complete ✓
- ☒ Project Proposal & Development Plan is finalized ✓
- ☒ Operational Governance Document is published ✓
- ☒ All technical documentation is up-to-date ✓

12. Future Roadmap

12.1 Version 2.0 Features

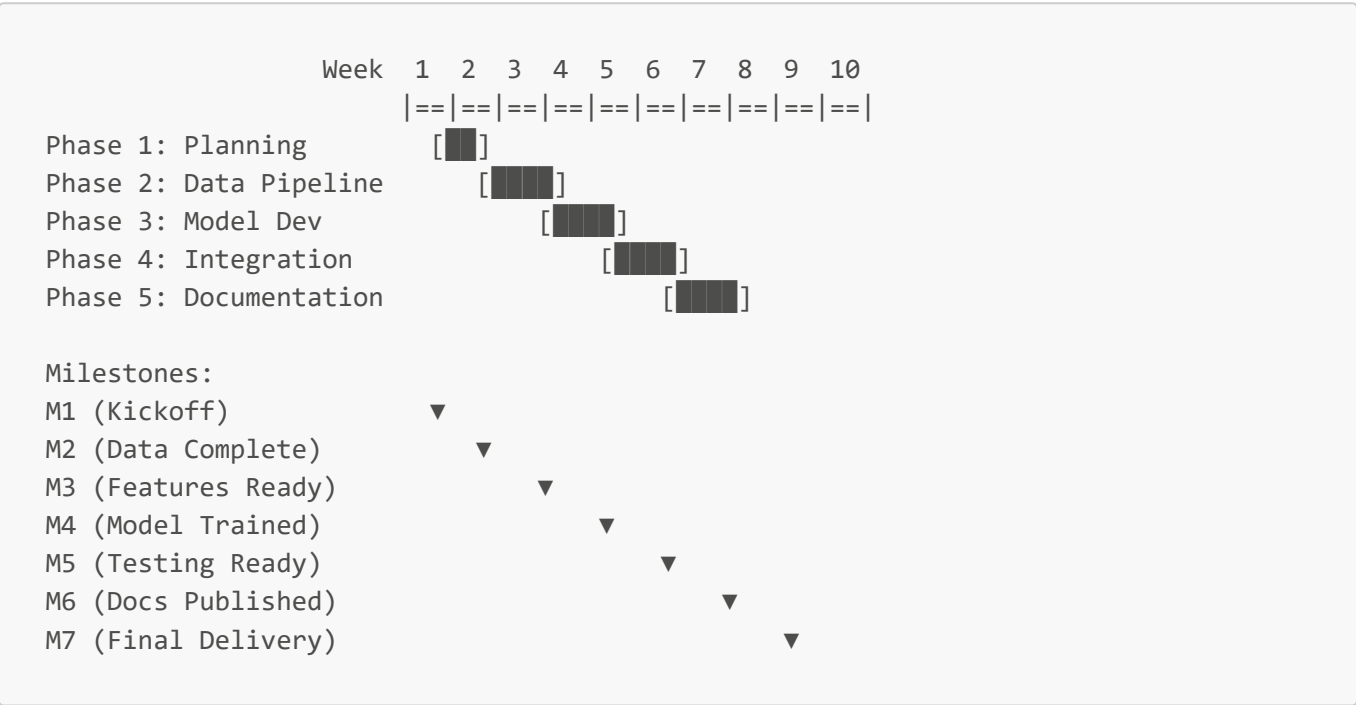
- Real-time stream processing with Kafka
- Multi-model ensemble (Isolation Forest, Autoencoder)
- Advanced alert notification (Email, Slack, webhooks)

- Enhanced dashboard features

12.2 Version 3.0 Features

- Cloud deployment (AWS/Azure/GCP)
- Kubernetes orchestration
- Distributed processing with Apache Spark
- Time-series database integration (InfluxDB)
- Experiment tracking with Neptune.ai

Appendix A: Gantt Chart



Appendix B: Backlog Prioritization

High Priority (Must Have) - ✓ Completed

- ✓ Data collection pipeline
- ✓ Feature engineering
- ✓ OCSVM model training
- ✓ Basic inference engine
- ✓ Core documentation

Medium Priority (Should Have) - ✓ Completed

- ✓ Advanced feature extraction
- ✓ Model hyperparameter optimization
- ✓ Comprehensive testing
- ✓ Performance benchmarking
- ✓ CI/CD automation

Low Priority (Could Have) - ✓ Implemented

- ✓ Dashboard prototype → Full Streamlit dashboards deployed
- ✓ Monitoring dashboards → Prometheus/Grafana stack
- Alert notification system (Future)
- Model explainability (Future)

Document Approval

Role	Name	Date
Project Manager	Francesca Craievich	2025-11-29
ML Engineer	Lucas Jakin	2025-11-29
Data Scientist	Francesco Rumiz	2025-11-29

Document Version History

Version	Date	Changes
1.0	2025-11-29	Initial proposal creation
1.1	2025-12-11	Updated milestones, added implemented features
1.2	2025-12-12	Added Project Evolution & Decision History, Agile methodology
2.0	2025-12-12	Major reorganization: focused on planning/management only, moved technical rationale to SSD, consolidated Agile methodology from Governance